

Atlas: Multi-Agent Research & Brief Generator

Manasi Mathkar · Wipro Junior FDE Pre-screening Assignment · May 2026

1. Use Case

Atlas takes a research question, such as “What is the current state of solid-state battery commercialization?”, and turns it into a sourced one-page brief. The final output includes key findings, source-backed analysis, risks, recommendations, a confidence score, and an agent trace that shows how the system reached its answer. Behind the scenes, five specialized agents collaborate through a LangGraph state machine. The workflow includes three security checkpoints and hard limits on autonomy, so the system remains auditable and bounded.

2. Multi-Agent Architecture

Atlas uses five agents arranged as a directed LangGraph workflow. The graph has one parallel research stage and one bounded review loop between the Critic and Writer. Importantly, the orchestrator controls the workflow; the LLMs do not decide which agent runs next. This keeps the system predictable, easier to debug, and safer than a fully open-ended agent conversation.

Communication: All agents read from and write to a shared typed GraphState defined with TypedDict. This state stores the user query, research plan, findings, draft brief, critique, confidence score, and guardrail decisions. For the parallel research step, the findings field uses a list-merge reducer so multiple Researcher agents can safely write results without overwriting each other. Agents do not call each other directly. Every transition is controlled by the graph router.

Execution model: The workflow is sequential at key decision points: Planner, Writer, and Critic. The Researcher agents run in parallel, with one Researcher per sub-question and a maximum of five sub-questions. After the Writer creates a draft, the Critic reviews it. If the Critic finds significant unsupported claims, the graph loops back to the Writer once for revision. This creates a useful adversarial review step without allowing the system to spiral into unlimited retries or uncontrolled agent behavior.

Agent	Role	Tools	Model
Planner	Decompose user query into ≤ 5 sub-questions	none	Claude Sonnet
Researcher ($\times N$)	Parallel web search + summarize per sub-Q	Tavily search (allowlisted)	Claude Sonnet
Writer	Synthesize sourced brief from findings	none	Claude Sonnet
Critic	Fact-check vs. sources; emit confidence score; loop if weak	none	Claude Sonnet
Security	Input / web-content / output checkpoints	regex + LLM judge	Claude Haiku

3. Security, Safety, and Guardrails

Atlas treats all external content as untrusted. For a research agent, the biggest risk is not just user prompt injection, but **indirect prompt injection** from fetched web pages. Since Tavily can return public web content, a malicious page could include instructions that try to manipulate the Writer. Atlas prevents this by treating source content as data, not commands. Atlas uses three guardrail checkpoints:

- **Input guardrail:** Checks the user query for prompt injection and jailbreak patterns. Clear violations are blocked. Ambiguous cases are reviewed by a Claude Haiku classifier.
- **Web-content guardrail:** Strips HTML with **bleach**, scans fetched sources for injection markers, and drops flagged sources before synthesis.
- **Output guardrail:** Scans the final brief for PII and secret-like patterns, including SSNs, credit cards, emails, phone numbers, and API-key shapes. Sensitive content is redacted before display.

Non-LLM controls: Atlas restricts tools and permissions by design. Only the Researcher can access Tavily, and no agent has shell, raw HTTP, or filesystem access. The workflow is capped at 5 sub-questions, 10 search calls, 1 critic retry, 4k tokens per agent, and a 120-second timeout. `structlog` records agent steps, model calls, searches, and guardrail decisions. In deployment, the Docker container runs as a non-root user, and secrets are injected through AWS Lightsail environment variables instead of being committed to Git.

4. Implementation Approach

Atlas is built with **Python 3.11, LangGraph, FastAPI, Claude, Tavily, and a custom streaming frontend**. LangGraph handles orchestration, FastAPI serves the backend, Server-Sent Events stream live progress updates, Tavily provides web search, and Claude powers the planning, research, writing, critique, and guardrail classification steps. I chose LangGraph because it makes the workflow explicit and auditable. Each agent step, security checkpoint, branch, and retry is represented in the graph, so the architecture diagram, audit trail, and code align closely.

Each run starts with a typed state and passes through the compiled graph. Agents are stateless functions that read from the shared state, write structured outputs back to it, and let the graph router decide the next step.

Reliability and deployment: Atlas uses a single `END` node, capped Critic-to-Writer retry, search limits, request timeouts, exponential backoff, and structured fallback objects so failed model or search calls do not crash the run. The frontend shows the final brief, agent trace, sources, confidence score, audit trail, and graph state for demo visibility. Testing uses `pytest` for guardrails, PII redaction, graph structure, injected source detection, and budget enforcement. Atlas is deployed with Docker on AWS Lightsail Containers, with `/healthz` checks, environment-based secrets, dependency pinning, `.env` exclusion, and non-root execution.

5. Use of AI/LLMs and Collaboration

Atlas uses LLMs only where reasoning or language understanding is useful:

- **Planner:** Breaks the user query into focused sub-questions.
- **Researcher:** Summarizes web findings for each sub-question.
- **Writer:** Synthesizes the sourced brief.
- **Critic:** Checks the draft against the sources and assigns a confidence score.
- **Security:** Uses Claude Haiku for lightweight classification when regex checks need review.

The LLMs do not control the workflow. They do not choose tools, decide routes, or continue loops. Those decisions are handled by deterministic Python code in the LangGraph router.

The main collaboration happens between the Writer and Critic:

- The Writer creates a sourced draft.
- The Critic checks for unsupported claims and weak reasoning.
- If the Critic finds more than one serious issue, the graph sends the draft back to the Writer once.
- If issues remain after the second pass, Atlas surfaces the limitations instead of looping again.

This gives Atlas a bounded review process instead of open-ended agent chat. Each agent has one role, one output schema, no memory across runs, and no independent tool choice. The tradeoff is less open-ended exploration, but the benefit is better reliability, auditability, and safety.

6. Project links and resources

Repository: github.com/manasimathkar/atlas-multi-agent

Live demo: wipro-manasi-jc3b1dk9p8244.us-west-2.cs.amazonlightsail.com